

PORTADA

Índice

CAPÍTULO 1: INTRODUCCIÓN	4
1.1.- Introducción del proyecto.....	5
1.2.- Propósito.....	5
1.3.- Objetivos del Proyecto	6
1.4.- Coste del proyecto	6
1.4.1. Costes de Desarrollo	6
1.4.2. Costes de Implantación	8
CAPÍTULO 2: ANÁLISIS DEL SISTEMA	9
2.1.- Introducción	10
2.2.- Análisis de requisitos	10
2.2.1. Requerimientos funcionales.....	10
2.2.2. Requerimientos no funcionales.....	11
2.3.- Casos de uso: Diagramas y Narrativas de Casos de Uso	11
CAPÍTULO 3: DISEÑO DEL SISTEMA	17
3.1.- Introducción.	18
3.2.- Diagrama de Clases.....	18
3.3.1. Diseño Lógico.....	19
3.3.2. Diseño Conceptual.....	20
3.4.- Diseño de la Interfaz	21
CAPÍTULO 4: IMPLEMENTACIÓN.....	23
4.1.- Introducción.	24
4.2.- Arquitectura cliente/servidor.....	24
4.3.- Lenguajes de Programación.	24

4.4.- Herramientas de Desarrollo.....	25
4.5.- Codificación	25
CAPÍTULO 5: PRUEBAS DE SOFTWARE	27
5.1.- Introducción.	28
5.2.- Técnicas de prueba.	28
5.2.1 Pruebas de caja blanca o enfoque estructural.....	28
5.2.2 Pruebas de caja negra o enfoque funcional.	28
CAPÍTULO 6: CONCLUSIONES	31
6.1.- Conclusiones	32
6.2.- Propuestas futuras (opcional).	32
CAPÍTULO 7: BIBLIOGRAFÍA Y REFERENCIAS	33
7.1.- Referencias bibliográficas.	34
ANEXO 1: MANUAL DE INSTALACIÓN	35
ANEXO 2: MANUAL DE USUARIO	37

CAPÍTULO 1: INTRODUCCIÓN

1.1.- INTRODUCCIÓN DEL PROYECTO

El presente proyecto tiene como finalidad el desarrollo de una aplicación web orientada a la gestión de un laboratorio dedicado a muestras de aguas de diversa tipología. En el citado sector del control y análisis de aguas, los técnicos realizan la toma de muestra y la medición de parámetros in situ, siendo necesario el correcto registro de parámetros, almacenamiento y posterior consulta de esta información.

Ante esta necesidad, surge la idea de crear una plataforma digital que permita registrar, consultar y gestionar muestras de forma eficiente, centralizando la información en una base de datos accesible desde cualquier dispositivo.

Además, el sistema incorpora herramientas complementarias como visualización de datos mediante gráficos, localización de los puntos de muestreo en Google Maps para su navegación y funcionalidades adaptadas a entornos con conectividad limitada, lo que aporta un valor añadido frente a métodos tradicionales basados en registros a papel.

El uso de este proyecto permite mejorar la trazabilidad de los datos, reducir errores humanos y optimizar el trabajo de los técnicos en campo.

1.2.- PROPÓSITO

El propósito de este proyecto es diseñar y desarrollar una aplicación web completa que facilite la gestión integral de muestras de agua, desde su registro hasta su análisis y visualización.

La aplicación permitirá a los usuarios:

- Registrar muestras con sus parámetros asociados.
- Consultar y editar muestras previamente almacenadas.
- Visualizar información relevante mediante gráficos dinámicos integrados.
- Acceder a herramientas adicionales como navegación a los puntos de muestreo y comunicación interna tipo chat.

El sistema permite al usuario poder registrar muestras in situ sin depender de la cobertura existente en ese instante, ya que en los casos de no disponibilidad de cobertura se guarda la muestra en local hasta que la conexión permita el registro de la misma en la base de datos.

1.3.- OBJETIVOS DEL PROYECTO

Los objetivos del proyecto son:

- Realizar un análisis de requisitos funcionales y no funcionales del sistema.
- Diseñar la arquitectura de la aplicación web basada en tecnologías de actualidad.
- Desarrollar una aplicación que permita el registro, consulta y gestión de muestras de agua.
- Implementar una base de datos relacional para el almacenamiento estructurado de la información.
- Integrar herramientas de visualización de datos mediante gráficos dinámicos.
- Desarrollar una interfaz de usuario sencilla, intuitiva y responsive.
- Implementar funcionalidades adicionales como localización de los puntos en maps y comunicación interna instantánea mediante chat de empleados.
- Desplegar la aplicación en un entorno cloud para su acceso remoto.
- Elaborar la memoria final del proyecto.

1.4.- COSTE DEL PROYECTO

Estimación del coste y esfuerzo del software y/ o hardware requerido para la realización del mismo. Podemos dividir en dos subapartados:

1.4.1. COSTES DE DESARROLLO

Costes para la realización del sistema. Se dividen en **Costes Recursos Informáticos** (costes hardware y software) y **Costes de personal**.

Costes de recursos informáticos

Para el desarrollo del proyecto se han utilizado herramientas de software mayoritariamente gratuitas o de código abierto, por lo que el coste económico directo es reducido:

- Entorno de desarrollo (Visual Studio Code): 0 €
- Node.js y Express: 0 €
- Base de datos MySQL: 0 €
- Sistema de control de versiones (GitHub): 0 €
- Plataforma de despliegue (Railway, plan gratuito): 0 €
- Open Data de organismos del Estado (Confederación Hidrográfica del Guadalquivir y Aemet): 0 €
- Panel de visualización inteligente de datos (Looker/Data Studio): 0 €

En cuanto al hardware, se ha utilizado un equipo personal ya disponible, por lo que no se consideran costes adicionales.

Costes de personal

El principal coste del proyecto corresponde al tiempo invertido en su desarrollo. Se estima:

- Análisis y diseño: 25 horas
- Desarrollo backend: 50 horas
- Desarrollo frontend: 40 horas
- Integraciones y despliegue: 15 horas
- Pruebas y correcciones: 20 horas
- Documentación: 15 horas

Total estimado: **165 horas**

Si se considera un coste medio de 15 €/hora:

Coste total de desarrollo: 2.475 €

1.4.2. COSTES DE IMPLANTACIÓN

Los costes de implantación del sistema son mínimos, ya que se utilizan servicios en la nube con planes gratuitos:

- Despliegue en Railway: 0 € (gratuito en versión de prueba de 30 días)
- Base de datos en la nube: incluida en Railway
- Servicios de visualización (Looker/Data Studio): 0 €

Por tanto, el coste de implantación es prácticamente nulo en la fase actual.

Coste total estimado del proyecto

- Coste de desarrollo: 2.475 €
- Coste de implantación: 0 €

Coste total: 2.475 €

CAPÍTULO 2: ANÁLISIS DEL SISTEMA

2.1.- INTRODUCCIÓN

En esta fase del proyecto se describe el proceso de desarrollo de la aplicación, incluyendo el análisis de requisitos, el diseño del sistema, la implementación y las pruebas realizadas.

En primer lugar, se ha llevado a cabo un análisis de las necesidades del sistema, identificando los requerimientos funcionales y no funcionales. Posteriormente, se ha diseñado la arquitectura de la aplicación, definiendo tanto la estructura de la base de datos como la organización del código.

La implementación se ha realizado mediante tecnologías web actuales, desarrollando tanto el backend como el frontend de la aplicación. Finalmente, se han realizado pruebas para verificar el correcto funcionamiento del sistema.

2.2.- ANÁLISIS DE REQUISITOS

El propósito del proyecto es desarrollar una aplicación web que permita la gestión de muestras de agua tomadas, garantizando el registro de parámetros asociados, consulta y visualización de los datos.

Para ello, el sistema debe cumplir una serie de requerimientos que definen su funcionamiento. Dichos requerimientos se dividen en funcionales y no funcionales

2.2.1. REQUERIMIENTOS FUNCIONALES.

Los requerimientos funcionales que se han identificado son los siguientes:

- El sistema permitirá el registro de usuarios y autenticación mediante inicio de sesión.
- El sistema diferenciará roles de usuario (administrador y técnico) con distintos permisos.
- El sistema permitirá la gestión de proyectos (solo administradores).
- El sistema permitirá registrar nuevas muestras de agua con sus parámetros asociados.
- El sistema permitirá consultar las muestras almacenadas en la base de datos.
- El sistema permitirá editar y actualizar la información de las muestras.
- El sistema permitirá eliminar registros si se precisa.

- El sistema integrará datos externos Open Data mediante servicios como Apps Script y los mostrará mediante gráficos integrados (Looker Studio).
- El sistema permitirá la visualización de puntos de muestreo en Google maps para su localización y navegación.
- El sistema incluirá un módulo de chat para comunicación interna entre empleados.

2.2.2. REQUERIMIENTOS NO FUNCIONALES

Los requerimientos no funcionales que se han identificado son los siguientes:

- Usabilidad: la interfaz debe ser sencilla y accesible desde distintos dispositivos (responsive).
- Rendimiento: el sistema debe responder de forma rápida en operaciones de registro y consulta.
- Escalabilidad: el sistema debe permitir futuras ampliaciones sin afectar su funcionamiento.
- Seguridad: se debe garantizar la autenticación de usuarios y la protección de sesiones.
- Mantenibilidad: el código debe estar estructurado y organizado para facilitar futuras modificaciones.
- Compatibilidad: la aplicación debe funcionar en los principales navegadores web.
- Fiabilidad: el sistema debe garantizar la integridad de los datos almacenados.

2.3.- CASOS DE USO: Diagramas y Narrativas de Casos de Uso

En este proyecto, los actores principales son:

- Usuario administrador
- Usuario técnico

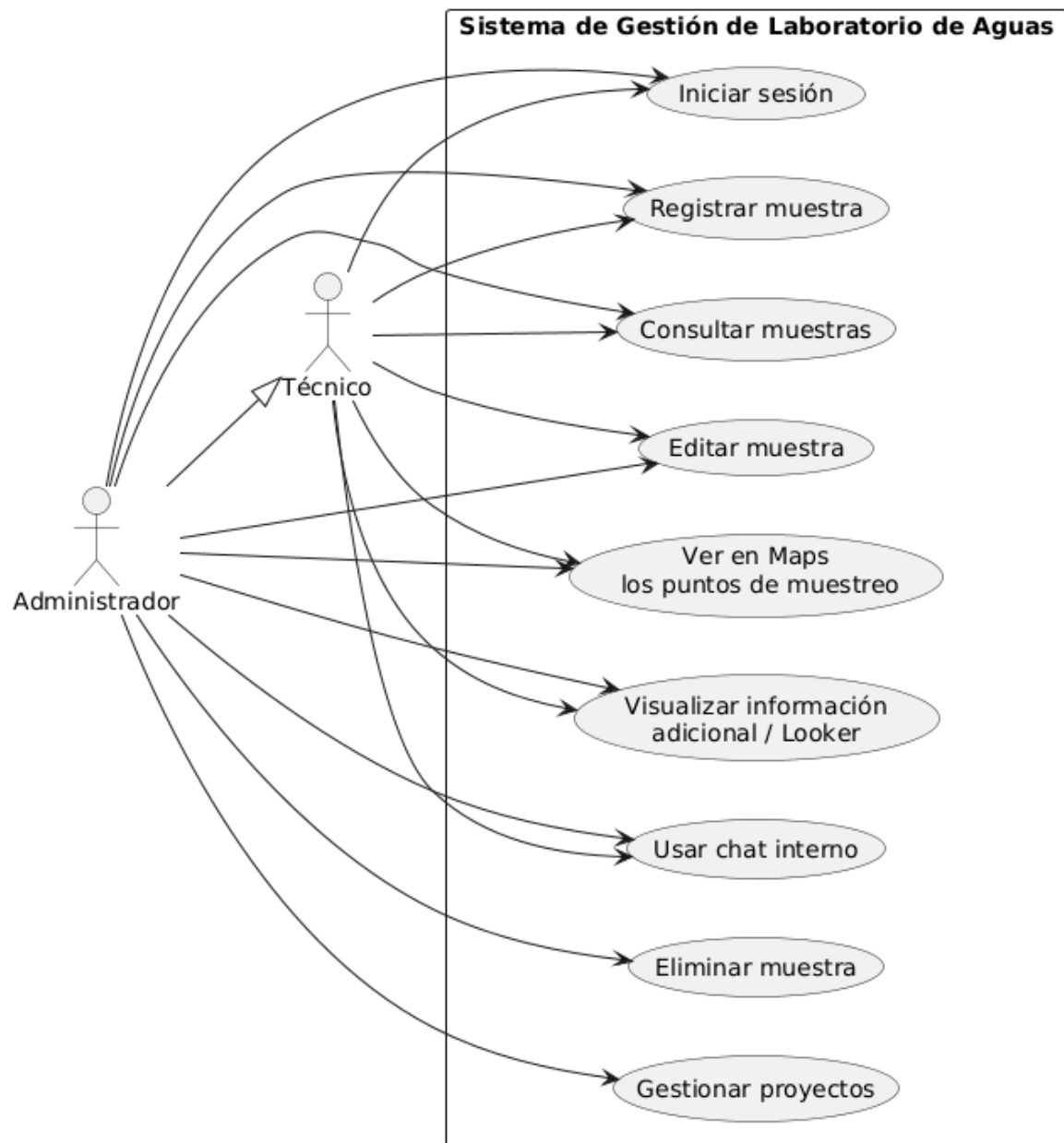
DIAGRAMA DE CASOS DE USOS

Ilustración. - Gestión integral del laboratorio.

a) Caso de uso 1: Registrar muestra

Nombre del caso de uso: Registrar muestra.

Actor Principal: Usuario técnico / administrador.

Condiciones de entrada: Usuario autenticado.

Flujo de eventos:

- El usuario accede a la sección “Nueva muestra”.
- El sistema muestra el formulario de registro.
- El usuario introduce los datos de la muestra.
- El usuario confirma el registro.
- El sistema valida los datos.
- El sistema almacena la muestra en la base de datos.
- El sistema muestra un mensaje de confirmación.

Camino alternativo:

- Si los datos son incorrectos, el sistema muestra un error y solicita corrección.

Condiciones de salida: La muestra queda registrada en la base de datos.

b) Caso de uso 2: Consultar muestras

Nombre del caso de uso: Consultar muestra.

Actor Principal: Usuario técnico / administrador.

Condiciones de entrada: Usuario autenticado.

Flujo de eventos:

- El usuario accede a la sección de consulta.
- El sistema muestra un listado de muestras.
- El usuario puede filtrar o buscar información.
- El sistema actualiza los resultados.

Camino alternativo:

- Si el filtro no coincide, el sistema no muestra ningún registro.

Condiciones de salida: El usuario visualiza las muestras solicitadas.

c) Caso de uso 3: Gestión de proyectos (administrador)

Nombre del caso de uso: Gestión de proyectos

Actor Principal: Usuario administrador.

Condiciones de entrada: Usuario autenticado como administrador.

Flujo de eventos:

- El usuario accede al módulo de proyectos.
- El sistema muestra los proyectos existentes.
- El usuario puede crear, editar o dar de baja/alta proyectos.

Condiciones de salida: Los cambios quedan guardados en el sistema.

d) Caso de uso 4: Visualizar información adicional

Nombre del caso de uso: Información adicional

Actor Principal: Usuario administrador/técnico

Condiciones de entrada: Usuario autenticado.

Flujo de eventos:

- El usuario accede a “Información adicional”.
- El sistema carga los paneles de visualización (Looker Studio).
- El usuario visualiza gráficos y datos

Condiciones de salida: Se muestran los gráficos correctamente.

e) Caso de uso 5: Ver ubicación de puntos de muestreo

Nombre del caso de uso: Navegación a los puntos de muestreo

Actor Principal: Usuario administrador/técnico

Condiciones de entrada: Usuario autenticado.

Flujo de eventos:

- El usuario accede a la sección “Navegación”.
- El sistema muestra el mapa con los puntos de muestreo.
- El usuario selecciona un punto.
- El sistema muestra la localización asociada.

- El usuario puede utilizar la información para desplazarse al punto de muestreo.

Condiciones de salida: El usuario visualiza la ubicación del punto de muestreo.

f) **Caso de uso 6: Comunicación interna**

Nombre del caso de uso: Usar chat interno

Actor Principal: Usuario administrador/técnico .

Condiciones de entrada: Usuario autenticado.

Flujo de eventos:

- El usuario accede a la sección “Chat”.
- El sistema muestra el contenedor para el chat.
- El usuario escribe un mensaje.
- El usuario envía el mensaje.
- El sistema muestra el mensaje en la conversación.

Condiciones de salida: El mensaje queda registrado y visible para los empleados que se encuentran en la vista chat.

g) **Caso de uso 7: Editar muestra**

Nombre del caso de uso: Editar muestra

Actor Principal: Usuario administrador/técnico .

Condiciones de entrada: Usuario autenticado y existe la muestra a editar.

Flujo de eventos:

- El usuario accede a la sección “Consulta de muestras”.
- El sistema muestra el listado de muestras registradas (usuario técnico solo las registradas por ese usuario)..
- El usuario selecciona una muestra concreta.
- El usuario pulsa la opción “Editar”.
- El sistema muestra el formulario con los datos actuales de la muestra.
- El usuario modifica los campos necesarios.

- El usuario confirma los cambios.
- El sistema actualiza la información en la base de datos.

Camino alternativo: Si el usuario cancela la edición, el sistema vuelve al listado sin guardar cambios.

Condiciones de salida: La muestra queda actualizada correctamente en el sistema.

h) **Caso de uso 8: Eliminar muestra**

Nombre del caso de uso: Eliminar muestra

Actor Principal: Usuario administrador/técnico .

Condiciones de entrada: Usuario autenticado y existe la muestra a eliminar.

Flujo de eventos:

- El usuario accede a la sección “Consulta de muestras”.
- El sistema muestra el listado de muestras registradas (usuario técnico solo las registradas por ese usuario).
- El usuario selecciona una muestra concreta.
- El usuario pulsa la opción “Eliminar”.
- El sistema solicita confirmación de la acción.
- El usuario confirma la eliminación.
- El sistema elimina la muestra de la base de datos.

Camino alternativo: Si el usuario cancela la acción, el sistema no elimina la muestra y vuelve al listado.

Condiciones de salida: La muestra queda eliminada del sistema.

CAPÍTULO 3: DISEÑO DEL SISTEMA

3.1.- INTRODUCCIÓN.

En el presente capítulo se aborda el diseño del sistema desarrollado, tomando como base los requisitos definidos en fases anteriores. El objetivo principal es definir una solución técnica que permita satisfacer las necesidades planteadas para la gestión integral de un laboratorio de muestras de aguas.

El diseño del sistema está orientado a la implementación mediante tecnologías web actuales. Se ha estructurado en varias fases clave que permiten organizar y comprender el funcionamiento interno de la aplicación:

- Diagrama de clases.
- Diseño de la base de datos.
- Diseño de la interfaz.

3.2.- DIAGRAMA DE CLASES.

Diagrama que describe la estructura de un sistema mostrando sus clases, atributos y las relaciones entre ellos.

En este proyecto, las clases principales se corresponden con los modelos definidos en la aplicación utilizando Sequelize. Entre las más relevantes se encuentran:

Usuario

- id
- nombre
- email
- contraseña
- rol (técnico / administrador)

Muestra

- id
- fecha
- ubicación
- parámetros físico-químicos
- observaciones
- id_usuario (relación con Usuario)

Proyecto

- id
- nombre
- descripción

Relaciones principales:

- Un proyecto puede tener empleados a varios usuarios (1:N)
- Un proyecto puede agrupar varias muestras (1:N)
- Un usuario puede registrar varias muestras (1:N)

3.3.- DISEÑO DE LA BASE DE DATOS.

El diseño de la base de datos define la estructura de almacenamiento de la información del sistema. En este caso, se ha utilizado una base de datos relacional MySQL, gestionada mediante el ORM Sequelize.

El objetivo de este diseño es garantizar la integridad de los datos, la consistencia en las relaciones y la eficiencia en consultas y operaciones.

3.3.1. DISEÑO LÓGICO.

En el diseño lógico se identifican las entidades principales del sistema y sus relaciones:

Entidades:

- Usuario
- Muestra
- Proyecto

Atributos relevantes:

- Identificadores únicos (id)
- Datos descriptivos (nombre, ubicación, parámetros)
- Claves externas (id_usuario, id_proyecto)

Relaciones:

- Usuario → Muestra (1:N)
- Proyecto → Muestra (1:N)
- Proyecto → Usuarios (1:N)

Cada entidad dispone de una clave primaria (id) que permite identificar de forma única cada registro dentro del sistema.

3.3.2. DISEÑO CONCEPTUAL.

El diseño se traduce en tablas dentro de la base de datos:

Tabla: usuarios

- id (INT, PK, AUTO_INCREMENT)
- nombre (VARCHAR)
- email (VARCHAR, UNIQUE)
- password (VARCHAR)
- rol (ENUM: técnico, administrador)

Tabla: muestras

- id (INT, PK)
- fecha (DATE)
- proyectoId (FK → proyectos.id)
- cada parametro (INT)
- observaciones (TEXT)
- usuario_id (FK → usuarios.id)

Tabla: proyectos

- id (INT, PK)
- nombre (VARCHAR)
- descripcion (TEXT)
- ubicación (TEXT)
- activo (Boolean)

Restricciones:

- Claves primarias en todas las tablas.
- Claves foráneas para mantener integridad referencial.
- Campos obligatorios en datos críticos.
- Validaciones gestionadas principalmente desde controller (backend).

3.4.- DISEÑO DE LA INTERFAZ.

Se ha desarrollado una interfaz web responsive utilizando Bootstrap, con plantillas renderizadas mediante EJS.

Características principales:

- Diseño adaptable a dispositivos móviles.
- Navegación clara e intuitiva.
- Acceso autenticación del usuario.

Estructura de pantallas:

1. Inicio (Sin autenticar)

- Inicio
- Contacto

2. Inicio (autenticado)

Pantalla principal con acceso rápido a:

- Nueva muestra
- Consulta
- Navegación
- Chat
- Información adicional (Looker)

3. Registro de muestras

Formulario para introducir datos de campo:

- Proyecto
- Parámetros físico-químicos
- Observaciones
- Foto del punto

4. Consulta de muestras

Listado con opciones de:

- Filtrado
- Editar
- Eliminar

5. Navegación

- Visualización de puntos de muestreo en Google maps.

5. Chat interno

- Sistema de mensajería instantánea entre empleados conectados.

6. Información adicional

Integración con Looker Studio mediante iframe:

- Administrador: datos de las muestras registradas y Open Data CHG
- Técnico: datos Open Data de predicción meteorológica (AEMET)

CAPÍTULO 4: IMPLEMENTACIÓN

4.1.- INTRODUCCIÓN.

En esta etapa se transforma el modelo conceptual y lógico en código fuente funcional, dando lugar a una aplicación web completamente operativa.

Durante esta fase se ha desarrollado tanto el backend como el frontend de la aplicación, integrando la base de datos y los servicios externos necesarios para su funcionamiento. Además, se ha preparado el sistema para su ejecución en distintos entornos, tanto local como en producción.

4.2.- ARQUITECTURA CLIENTE/SERVIDOR

La aplicación desarrollada sigue una arquitectura cliente/servidor. El cliente es el navegador web del usuario, desde donde se realizan las peticiones. El servidor es la aplicación desarrollada con Node.js y Express, que procesa las solicitudes y devuelve las respuestas.

El funcionamiento es el siguiente:

- El usuario interactúa con la interfaz (cliente)
- El cliente envía una petición HTTP al servidor
- El servidor procesa la petición
- Accede a la base de datos (MySQL)
- Ejecuta la lógica
- El servidor devuelve una respuesta (HTML renderizado con EJS)
- El cliente muestra la información al usuario

4.3.- LENGUAJES DE PROGRAMACIÓN.

Para el desarrollo del proyecto se han utilizado los siguientes lenguajes:

-JavaScript

Es el lenguaje principal del proyecto. Se ha utilizado tanto en el backend (Node.js) como en el frontend para la interacción con el usuario. Gestiona la lógica de la aplicación, las peticiones al servidor y la manipulación de datos.

-HTML

Se ha utilizado para definir la estructura de las páginas web. A través de plantillas EJS, se generan vistas dinámicas que se envían al cliente.

-CSS

Se ha utilizado para el diseño visual de la aplicación, apoyándose en el framework Bootstrap para facilitar la creación de una interfaz responsive y más actual.

-SQL

Se ha utilizado para la gestión de la base de datos MySQL, definiendo las tablas, relaciones y consultas de datos registrados.

4.4.- HERRAMIENTAS DE DESARROLLO.

Para la realización del proyecto se han empleado diversas herramientas de desarrollo:

- Visual Studio Code: entorno de desarrollo integrado (IDE), utilizado para escribir y organizar el código.
- Node.js: entorno de ejecución para JavaScript en el servidor.
- Express: framework para la aplicación web en Node.js
- Sequelize: ORM utilizado para interactuar con la base de datos usando JavaScript en lugar de consultas SQL MySQL.
- MySQL: sistema gestor de bases de datos relacional.
- Git y GitHub: control de versiones y almacenamiento del código fuente.
- Railway: plataforma cloud utilizada para el despliegue de la aplicación en producción.
- Looker Studio: herramienta de visualización de datos inteligente integrada mediante iframe.
- XAMPP (entorno local): para desarrollo con base de datos local.

4.5.- CODIFICACIÓN.

Todo el código desarrollado se encuentra disponible en el repositorio del proyecto en GitHub, permitiendo la revisión pertinente.

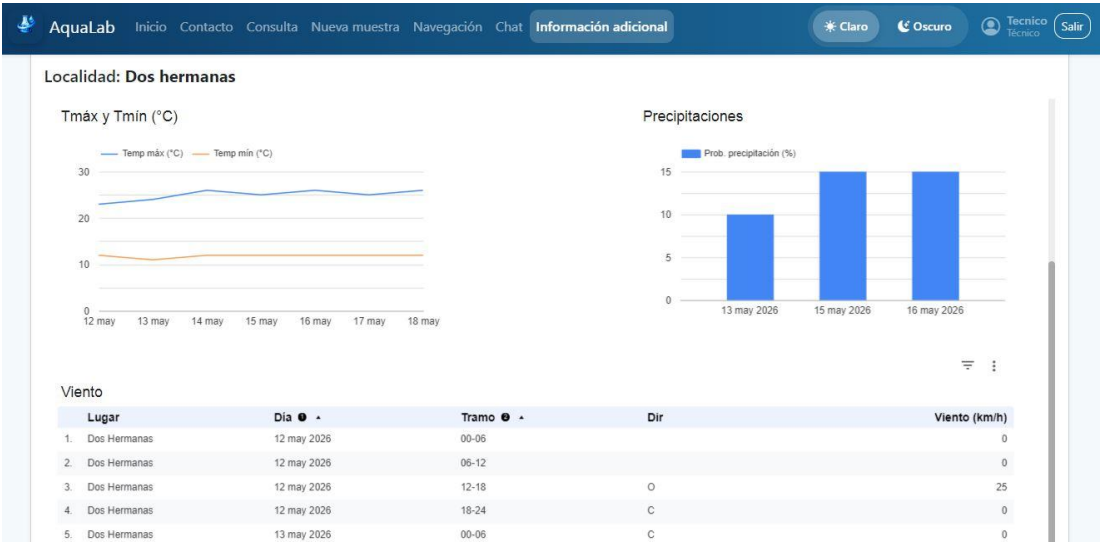
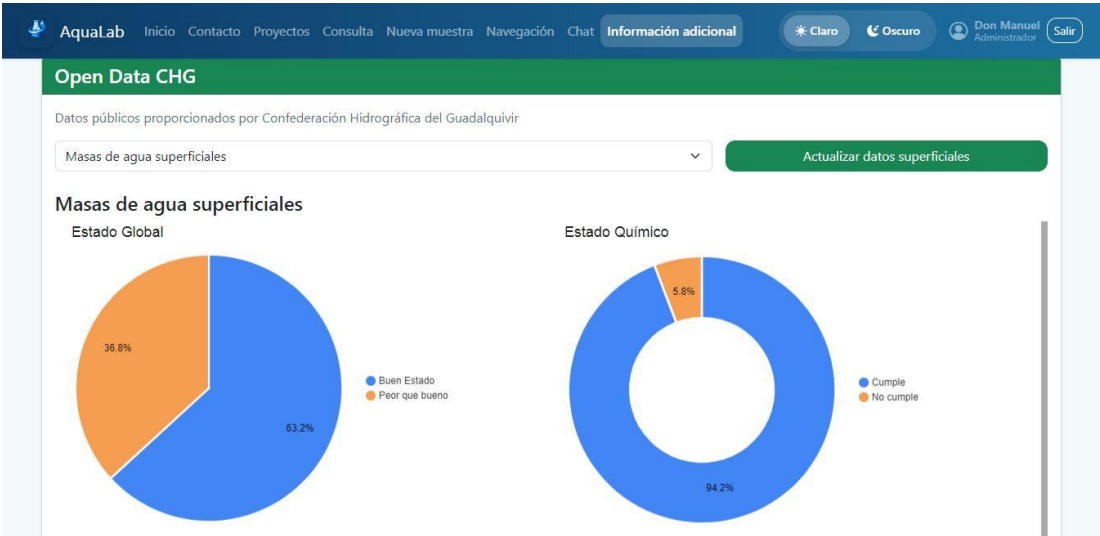
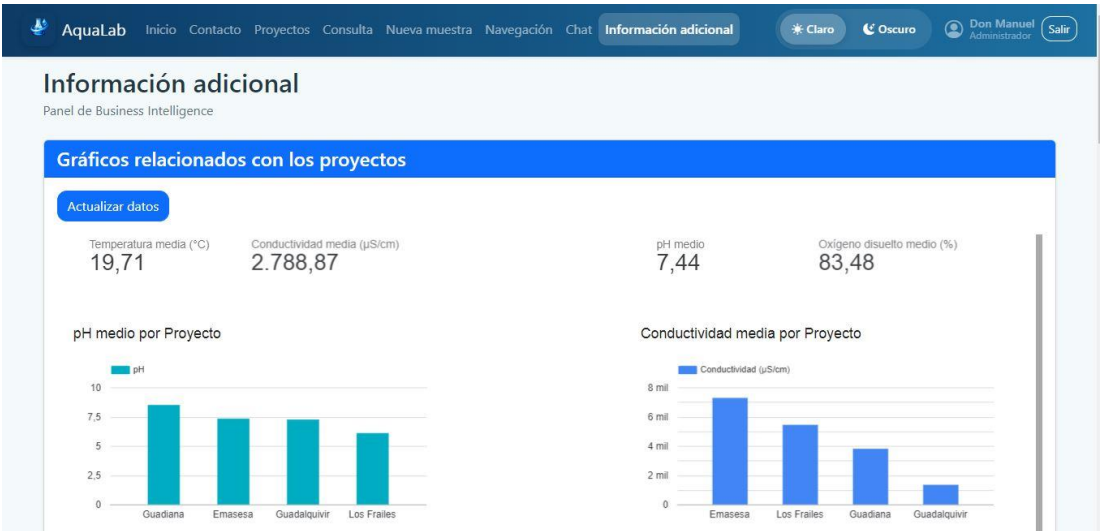


Ilustración. - Panel de Looker Studio.

CAPÍTULO 5: PRUEBAS DE SOFTWARE

5.1.- INTRODUCCIÓN.

En este capítulo se describen las pruebas realizadas sobre la aplicación desarrollada, incluyendo tanto pruebas automatizadas como pruebas funcionales manuales. El objetivo es validar el comportamiento del sistema en diferentes escenarios y asegurar su correcto funcionamiento en distintos entornos y dispositivos.

5.2.- TÉCNICAS DE PRUEBA.

Las pruebas de software pueden clasificarse en pruebas de caja blanca y pruebas de caja negra.

Las pruebas de caja blanca se centran en la estructura interna del código.

Las pruebas de caja negra evalúan el comportamiento del sistema desde el punto de vista del usuario.

5.2.1 PRUEBAS DE CAJA BLANCA O ENFOQUE ESTRUCTURAL.

En este proyecto se han tenido en cuenta los siguientes aspectos durante el desarrollo:

- Validación de rutas y controladores en el backend (Express).
- Comprobación del flujo de datos entre modelos, vistas y controladores (MVC).
- Gestión correcta de errores en operaciones con la base de datos.
- Control de autenticación y sesiones de usuario.

Estas comprobaciones se han realizado de forma continua durante la fase de desarrollo.

5.2.2 PRUEBAS DE CAJA NEGRA O ENFOQUE FUNCIONAL.

Las pruebas de caja negra evalúan el sistema desde el punto de vista del usuario, sin tener en cuenta su codificación. Con ello, se pretende comprobar que la aplicación cumple correctamente con su funcionalidad.

En este proyecto se han realizado dos tipos principales de pruebas de caja negra:

1. Pruebas automatizadas con Playwright

Se han implementado pruebas automatizadas utilizando Playwright, una herramienta que permite simular la interacción del usuario con la aplicación a través del navegador.

Estas pruebas han permitido verificar automáticamente funcionalidades clave como:

- Inicio de sesión de usuarios.
- Registro de nuevas muestras.
- Navegación entre diferentes secciones de la aplicación.
- Acceso a funcionalidades según el rol del usuario.
- Correcta carga de páginas y elementos.

2. Pruebas funcionales manuales

Además de las pruebas automatizadas, se han realizado pruebas manuales para validar el comportamiento real de la aplicación en distintos entornos. Estas pruebas se han llevado a cabo en PC y dispositivos móviles, verificando el diseño responsive.

Se han comprobado:

- Correcta visualización de la interfaz
- Funcionamiento de formularios (registro, edición, eliminación)
- Navegación entre páginas
- Visualización de gráficos (Looker Studio)
- Adaptación del diseño a diferentes tamaños de pantalla

Q s:passed

All 4 ✓ Passed 4 Failed 0 Flaky 0 Skipped 0 ⚙️

« previous next »

Login

deja escribir email y contraseña

login.spec.js:11 1.5s

✓ Run

Test Steps

Q Filter steps

> ✓ Before Hooks	206ms
> ✓ Navigate to "/login" — login.spec.js:12	560ms
> ✓ Fill "usuario@aqualab.com" locator("#email") — login.spec.js:13	124ms
> ✓ Fill "1234" locator("#password") — login.spec.js:14	35ms
> ✓ Expect "toHaveValue" locator("#email") — login.spec.js:15	20ms
> ✓ Expect "toHaveValue" locator("#password") — login.spec.js:16	10ms
> ✓ After Hooks	525ms

> Executed in Worker #0

Q Search tests

All 4 ✓ Passed 3 ✗ Failed 1 Flaky 0 Skipped 0 ⚙️

next »

Nueva muestra

permite crear una muestra nueva

muestras.spec.js:41 7.4s

✗ Run

Errors

Error: expect(page).toHaveURL(expected) failed

Expected pattern: /consult?success=created/i
Received string: "http://localhost:3000/nueva"
Timeout: 5000ms

Call log:

- Expect "toHaveURL" with timeout 5000ms
- 9 x unexpected value "http://localhost:3000/nueva"

```
50 |     await page.fill('#observaciones', 'Alta de muestra de prueba');
51 |     await page.getByRole('button', { name: 'guardar/i' }).click();
> 52 |     await expect(page).toHaveURL(/consult?success=created/i);
    |                        ^
53 |     await expect(page.locator('.alert-success')).toContainText(/Muestra creada/i);
54 |   });
55 |   test('tiene botón para volver a consulta', async ({ page }) => {
    at C:\Users\LunaFG\Desktop\test-PFC\tests\muestras.spec.js:52:24
```

Ilustración. - Pruebas automatizadas con Playwright

CAPÍTULO 6: CONCLUSIONES

6.1.- CONCLUSIONES

Se ha desarrollado una aplicación web para la gestión de un laboratorio de muestras de agua, permitiendo su registro, consulta, edición y visualización de datos de forma eficiente.

El sistema implementado ofrece una solución para el trabajo de técnicos en campo, facilitando el registro de parámetros in situ incluso en condiciones con conectividad limitada. Además, la integración de herramientas como la visualización de datos mediante Looker Studio permitir análisis gráfico de la información registrada, así como de información proveniente de apis rest de organismos públicos (OPEN DATA) que pudiera ser de utilidad.

Desde el punto de vista técnico, el uso de tecnologías como Node.js, Express, Sequelize y MySQL ha permitido construir una arquitectura basada en el modelo cliente-servidor con herramientas de actualidad. Asimismo, el despliegue en un entorno real (Railway trial) ha permitido validar el funcionamiento del sistema en producción.

Finalmente, la realización de pruebas automatizadas y funcionales ha contribuido a garantizar el correcto funcionamiento de la aplicación en distintos dispositivos y escenarios.

6.2.- PROPUESTAS FUTURAS (OPCIONAL).

Como posibles mejoras y ampliaciones del sistema, se plantean las siguientes:

- Mejora del sistema offline, permitiendo sincronización automática más avanzada.
- Ampliación del módulo de análisis de datos con más gráficos y filtros avanzados.
- Registros de muestras con más de una foto por registro.
- Persistencia de mensajes anteriores en el chat interno.

CAPÍTULO 7: BIBLIOGRAFÍA Y REFERENCIAS

7.1.- REFERENCIAS BIBLIOGRÁFICAS.

Para el desarrollo del presente proyecto se han consultado diversas fuentes de información: documentación oficial y recursos formativos online.

- MDN web
- OpenWebinars: Plataforma de formación donde se realizó el curso de Node.js, Express y Socket.io aplicado en el proyecto.
- Bootstrap web
- W3Schools web
- Node.js web
- Express web
- Sequelize web
- MySQL web
- Playwright web
- Looker Studio web
- Railway web.
- GitHub.

ANEXO 1: MANUAL DE INSTALACIÓN

1. Introducción

En este apartado se describen los pasos necesarios para instalar y ejecutar la aplicación de gestión de laboratorio de muestras de agua en un entorno local.

2. Requisitos previos

Antes de comenzar, es necesario disponer de los siguientes elementos instalados:

- Node.js (incluye npm)
- MySQL
- Git
- Visual Studio Code o IDE similar

3. Descarga del proyecto

Clonar el repositorio desde GitHub:

```
"git clone https://github.com/jdomfern/tfg-laboratorio.git"
```

4. Instalación de dependencias

Acceder a la carpeta del proyecto en consola/PowerShell y ejecutar "npm install". Este comando instalará todas las dependencias necesarias.

5. Configuración de la base de datos

Crear una base de datos en MySQL.

- Configurar el archivo .env con los datos de conexión:
- DB_HOST=127.0.0.1
- DB_PORT=3306
- DB_NAME=(elección del usuario)
- DB_USER=(elección del usuario)
- DB_PASSWORD=(elección del usuario)
- NODE_ENV=development

6. Ejecución de la aplicación

Para iniciar el servidor en entorno local ejecutar en consola/PowerShell " npm start". Una vez iniciado, la aplicación estará disponible en "http://localhost"

ANEXO 2: MANUAL DE USUARIO

1. Introducción

Este manual explica de forma sencilla el uso de la aplicación web de gestión de laboratorio de muestras de agua.

2. Acceso a la aplicación

Para acceder a la aplicación, el usuario debe abrir la dirección web correspondiente. Sin autenticación solo se muestra la página de inicio y la información de contacto del laboratorio. El usuario puede iniciar sesión con sus credenciales. Si no dispone de ellas, deberá registrarse previamente con un email corporativo (ej. @aqualab.com)

Una vez dentro, se muestra la pantalla principal con las opciones disponibles según el tipo de usuario.

3. Pantalla de inicio

Desde la pantalla principal se puede acceder a:

- Nueva muestra
- Consulta de muestras
- Navegación
- Chat
- Información adicional
- Proyectos (solo para administradores)

4. Registrar una nueva muestra

Para registrar una muestra:

- Pulsar en Nueva muestra
- Rellenar los datos recogidos en el formulario
- Añadir observaciones o fotografías si corresponde
- Guardar la muestra

El sistema almacenará la información en la base de datos.

5. Consultar muestras

En la sección Consulta, el usuario puede ver sus muestras registradas (todas si tiene el rol de administrador):

- Buscar muestras
- Revisar sus datos de parámetros
- Editar información
- Eliminar el registro

6. Navegación

La sección Navegación permite consultar la ubicación en Google maps de los puntos de muestreo y abrir dicha aplicación para la navegación del técnico hacia el punto.

7. Chat interno

El apartado Chat permite la comunicación interna entre usuarios presente en esa vista. El usuario puede escribir y enviar mensajes que serán visibles para el resto de los empleados.

8. Información adicional

En esta sección se muestran gráficos y datos de interés mediante paneles de Looker Studio. Estos gráficos permiten visualizar información relacionada con las muestras y otra información adicional que pudiera ser de interés (datos de Confederación Hidrográfica del Guadalquivir CHG y Agencia Española de Meteorología AEMET).

9. Gestión de proyectos

Esta opción está disponible únicamente para usuarios con rol de administrador. Permite crear y gestionar proyectos a los que se asociará las muestras de aguas registradas.

10. Cierre de sesión

Para salir de la aplicación, el usuario debe pulsar la opción Salir en el menú superior.